

Lecture 03: Conditionals and Expressions

Introduction to Programming with Kotlin

Agenda

1. Conditional operator: `if` / `else`
2. Combining conditions (Part 2)
3. `if` and `when` as expressions

Making decisions

Programs often choose different actions based on data.

- A **condition** is an expression that is true or false (type Boolean)
- `if` runs a block when the condition is true
- `else` covers the other case

Comparison operators

Operator	Meaning
==	equal to
!=	not equal to
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal

Common mistake

Do not confuse = (assignment) with == (equality).

Basic if / else

```
1 fun main() {  
2     print("Enter score: ")  
3     val score = readln().toInt()  
4     if (score >= 50) {  
5         println("Pass")  
6     } else {  
7         println("Fail")  
8     }  
9 }
```

Braces { } mark the blocks. Prefer always using braces, even for one-liners.

else if chains

```
1 fun main() {  
2     val temp = readln().toInt()  
3     if (temp < 0) {  
4         println("Freezing")  
5     } else if (temp < 15) {  
6         println("Cold")  
7     } else if (temp < 25) {  
8         println("Mild")  
9     } else {  
10        println("Warm")  
11    }  
12 }
```

Conditions are checked top to bottom; the first true branch wins.

Nested `if` (use sparingly)

```
1 fun main() {  
2     val age = readln().toInt()  
3     val hasTicket = readln() == "yes"  
4     if (age >= 18) {  
5         if (hasTicket) {  
6             println("Enter")  
7         } else {  
8             println("Buy a ticket")  
9         }  
10    } else {  
11        println("Too young")  
12    }  
13 }
```

Nesting deep trees is hard to read — combining conditions often helps.

Quiz 1 — if / else

Quiz

For `val x = 10`, which branch runs?

`if (x > 10) A else if (x >= 10) B else C`

1. A
2. B
3. C
4. A and B

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. B

$x > 10$ is false; $x \geq 10$ is true, so branch B runs. Only one branch of an if/else if chain executes.

Logical operators

Combine Booleans with:	Operator	Name	Idea	Kotlin uses short-circuit
	&&	and	both must be true	
		or	at least one true	
	!	not	flips true/false	

evaluation: if the result is already known, the rest is not evaluated.

And, or, not

```
1 fun main() {  
2     val age = readln().toInt()  
3     val hasId = readln() == "yes"  
4  
5     if (age >= 18 && hasId) {  
6         println("Adult with ID")  
7     }  
8     if (age < 13 || age > 65) {  
9         println("Discount age")  
10    }  
11    if (!(age >= 18)) {  
12        println("Under 18")  
13    }  
14 }
```

Truth tables (quick)

`&&` (and)

- $T \ \&\& \ T \rightarrow T$
- $T \ \&\& \ F \rightarrow F$
- $F \ \&\& \ T \rightarrow F$
- $F \ \&\& \ F \rightarrow F$

`||` (or)

- $T \ || \ T \rightarrow T$
- $T \ || \ F \rightarrow T$
- $F \ || \ T \rightarrow T$
- $F \ || \ F \rightarrow F$

`!true` is `false`; `!false` is `true`.

Readable ranges

Check that a value lies in an interval:

```
1 fun main() {  
2     val n = readln().toInt()  
3     // Inclusive range check with &&  
4     if (n >= 1 && n <= 10) {  
5         println("Between 1 and 10")  
6     }  
7     // Kotlin also has 'in' (more later with ranges)  
8     if (n in 1..10) {  
9         println("Also between 1 and 10")  
10    }  
11 }
```

Try this

Validate that a password length is at least 8 *and* the user typed yes to confirm.

Operator precedence (practical)

- Comparisons bind tighter than `&&` / `||`
- `&&` binds tighter than `||`
- When unsure, add parentheses

Examples

```
a > 0 && b > 0 || c > 0
```

```
means (a > 0 && b > 0) || c > 0
```

```
Prefer: (a > 0 && b > 0) || (c > 0)
```

Quiz 2 — Combining

Quiz

When is `age >= 18 && hasTicket true`?

1. Only when age is at least 18
2. Only when `hasTicket` is true
3. When both are true
4. When either is true

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. When both are true

`&&` requires both sides to be true. Use `||` if either condition is enough.

Statements vs expressions

In Kotlin, many control structures are **expressions**: they produce a value.

- You can assign the result of `if` to a variable
- Same idea with `when`
- This reduces duplicated `println` / assignment code

if as an expression

```
1 fun main() {  
2     val score = readln().toInt()  
3     val result = if (score >= 50) "Pass" else "Fail"  
4     println(result)  
5  
6     val max = if (score > 100) 100 else score  
7     println("Capped: $max")  
8 }
```

When used as an expression, else is required (both branches must produce a value).

Blocks as expression bodies

```
1 fun main() {  
2     val x = readln().toInt()  
3     val label = if (x % 2 == 0) {  
4         println("Even path")  
5         "even"  
6     } else {  
7         println("Odd path")  
8         "odd"  
9     }  
10    println(label)  
11 }
```

The last expression in the block is the value of that branch.

when as a better multi-branch

```
1 fun main() {  
2     val code = readln().toInt()  
3     val mood = when (code) {  
4         1 -> "Happy"  
5         2 -> "Okay"  
6         3, 4 -> "Tired"  
7         else -> "Unknown"  
8     }  
9     println(mood)  
10 }
```

when replaces long else if chains for discrete cases.

when with conditions

```
1 fun main() {  
2     val n = readln().toInt()  
3     val kind = when {  
4         n < 0 -> "negative"  
5         n == 0 -> "zero"  
6         n % 2 == 0 -> "positive even"  
7         else -> "positive odd"  
8     }  
9     println(kind)  
10 }
```

Subject-less when { } checks Boolean conditions in order.

when with ranges (preview)

```
1 fun main() {  
2     val score = readln().toInt()  
3     val grade = when (score) {  
4         in 90..100 -> "A"  
5         in 80..89  -> "B"  
6         in 70..79  -> "C"  
7         in 60..69  -> "D"  
8         else      -> "F"  
9     }  
10    println(grade)  
11 }
```

Ranges are covered in depth in Lecture 04.

Quiz 3 — Expressions

Quiz

Which is valid Kotlin?

1. `val s = if (x > 0) "pos"`
2. `val s = if (x > 0) "pos" else "nonpos"`
3. `val s = when (x) { 1 -> "one" }` without else, for arbitrary `Int x`
4. `val s = if = (x > 0) "pos" else "no"`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. `val s = if (x > 0) "pos" else "nonpos"`

An `if` expression needs `else`. A `when` expression over an `Int` needs to be exhaustive (usually via `else`). Option 4 is invalid syntax.

Summary

- `if / else if / else` choose branches
- Combine with `&&`, `||`, `!`
- Prefer parentheses for clarity
- `if` and `when` can produce values (expressions)
- Use `when` for multi-way matching

Looking ahead

Next: collections, repeat / while, for and size, ranges, and do-while with split.